PROJECT GUIDE

THE

Hoop Ledger

An NBA forecasting system, end to end: what it is, how it works, what the numbers say, and how to run it.

5,390	65.1%	11.0	0.217
GAMES GRADED	WINNERS PICKED	PTS MARGIN MISS	BRIER SCORE

Mounir Arayssi · thehoopledger.com · github.com/MounirArayssi/NBA-Predictor

Data through 2026-11-08 · written November 9, 2026

Contents

1	The project in one page	2
2	Motivation: how much do the numbers reveal?	3
3	System overview	4
4	Tech stack	5
5	Data sources and the database	6
6	The 21 inputs (feature engineering)	8
7	The model	10
8	From margin to win probability	11
9	Forecast timing and the official record	12
10	How the model is tested	13
11	Results: all the numbers	14
12	What the numbers reveal	17
13	Experiments and decisions	20
14	Explaining a forecast	21
15	The website	22
16	Repository layout	24
17	Programming deep dives	25
18	Setting it up from scratch	32
19	Daily operations runbook	33
20	Retraining and changing the model	34
21	Updating and deploying the website	35
22	Maintainability: known issues and gotchas	36
23	Limitations and roadmap	37
A	Explaining it in an interview	38
B	Glossary	40
C	Numbers cheat sheet	41

PART I · THE PROJECT

1 The project in one page

The Hoop Ledger predicts the final score of every NBA game before tip-off, turns that into a winner and a win probability, saves the forecast, and grades it against the result once the game ends. Every forecast and every grade is published on a public website, **thehoopledger.com**.

Under the hood it is a Python data pipeline and machine-learning model backed by PostgreSQL, plus a static website generated from a read-only export of that database. The model uses **basketball information only**: results, box scores, the official injury report and the schedule. Betting lines are deliberately *not* inputs, so the model can later be compared with the market honestly.

5,390 games graded (Oct 2022 – Nov 2026)	**65.1%** winners picked	**55.6%** if you always picked the home team
11.0 pts average miss on the margin	**0.217** Brier score (0.25 = coin flip)	**98–48** 2026-27 so far (67.1%)

The pieces

- **Data pipeline** (Python): pulls schedules, scores and box scores from NBA Stats, parses the league's official injury-report PDFs, and stores everything in PostgreSQL.
- **Features**: 21 numbers per game, all known before tip-off: a margin-of-victory Elo rating, season scoring margin, 10-game offensive/defensive ratings and pace, points per game of injured rotation players, rest, and playoff-series state.
- **Model**: two gradient-boosted regression models (scikit-learn), one for the home score and one for the away score. The margin and total follow from those; a logistic curve turns the margin into a win probability.
- **Evaluation**: a walk-forward backtest — each season predicted by a model trained only on earlier seasons, with that day's injury report — plus the live record. 5,390 games in all.
- **Explainability**: exact per-input contributions (TreeSHAP) for forecasts whose model can be rebuilt, and every input traced back to the raw box-score stats behind it.
- **Website** (Astro, static): forecasts, results, team pages, a "what wins games" study, performance pages and a research notebook. Hosted on Cloudflare Pages.

The 20-second version

“I built a model that forecasts every NBA game before tip-off from team strength, recent form, injuries and rest. Every forecast is logged and graded in public. Over 5,390 games it picks the winner 65% of the time against a 56% home-team baseline, and its win probabilities are well calibrated. The interesting part is how little beyond team strength actually helps — and how much of a single game is just noise.”

2 Motivation: how much do the numbers reveal?

The project started from a simple question: **if you know everything the box scores say about two teams, how well can you actually predict the game?** Basketball is covered in statistics. The question is how much of a single result they explain, and which ones carry real signal.

Three findings frame everything else in this guide:

1. A good model beats the obvious baseline, but not by a landslide

Home teams won **55.6%** of the 5,390 graded games. The model picks **65.1%**. That ~10-point edge is real and stable across seasons (Section 11), but it shows the ceiling: in close matchups (1,954 games with a projected margin under 3) the model is right only **54.9%** of the time; with a projected margin of 10+ it is right **83.7%**.

2. Most of a single game is noise

Actual home margins have a standard deviation of **15.5 points**. The model's forecast margins have a standard deviation of only **6.8**. A forecast is an average outcome, so it *should* be narrower, but the gap means a typical game's result lands about 11 points away from even a well-informed expectation. Shooting variance, foul trouble, garbage time and late scratches swamp everything known beforehand.

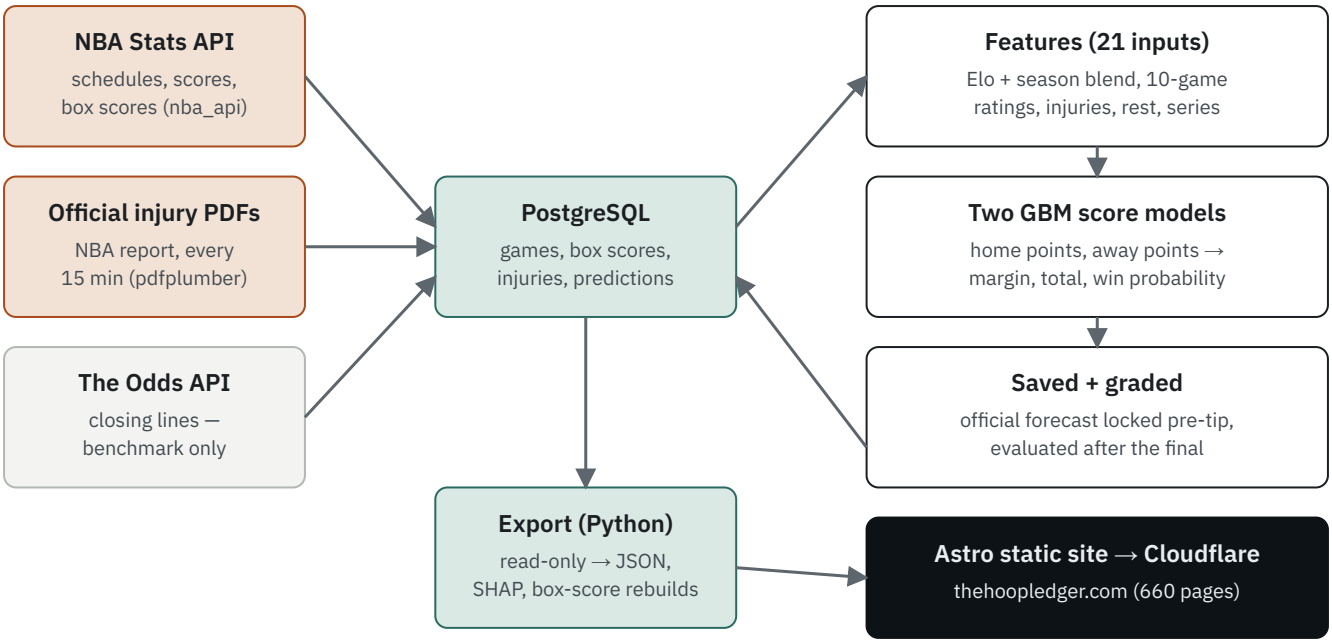
3. Few numbers matter; the rest re-describe them

An early version used 42 inputs (shooting splits, rebound rates, matchup products, scoring variance). Cutting to 21 — measuring team strength well once (Elo + season margin) and adding only what strength cannot see (injuries, rest, series) — **improved every metric out of sample** (Section 13). Across the full record, the single best predictor of the final margin is the pregame Elo gap (correlation 0.42); free-throw rate barely registers (0.08).

> **Design principles that came out of this**
>
> - **Honesty over flattery**: no betting lines as inputs, no after-the-fact forecasts in the record, every miss shown.
> - **Leakage paranoia**: every input must be knowable before tip-off; the backtest replays each day's injury report.
> - **Fewer, better features**: a feature earns its place only by improving a walk-forward test.
> - **Explain, don't assert**: show what the model received and exactly how it used it.

3 System overview

Data flows one way: from public sources into PostgreSQL, through feature building and the models, into saved forecasts, and out through a read-only export into a static website.



End-to-end flow. Orange: external sources. Teal: storage and export. The website never touches the database directly.

A forecast's life cycle

1. **Ingest** (daily): new results and box scores are fetched; rolling 10-game team stats and player stats are recomputed; today's schedule is loaded.
2. **Safety check**: before predicting, the pipeline confirms each team's previous game is final. If not, it re-fetches and fails loudly rather than forecast from stale data.
3. **Features**: Elo and season margin are replayed from every final game since October 2021; the latest official injury report is downloaded and parsed; rest and series state are computed.
4. **Predict**: the two score models produce home and away points; the margin becomes a win probability; the forecast (with its exact 21-number input vector) is saved with a timestamp.
5. **Lock**: the first forecast saved before tip-off is the official one; it can only be replaced within the final hour before tip; nothing saved after tip-off becomes official.
6. **Grade**: once the final is in, the forecast is scored (winner correct, score and margin errors).
7. **Publish**: an export script writes JSON; Astro builds ~660 pages; Wrangler uploads them to Cloudflare.

4 Tech stack

LAYER	TECHNOLOGY	USED FOR
Language	Python 3.14	Pipeline, features, models, evaluation, export
Data	pandas 3.0, NumPy 2.4	Feature tables, rolling windows, metrics
Machine learning	scikit-learn 1.8	`GradientBoostingRegressor` score models; `LogisticRegression` win-probability curve
Explainability	SHAP 0.54 (TreeExplainer)	Exact per-input contributions for tree models
Database	PostgreSQL 18, SQLAlchemy 2.0, psycopg2	All data and every forecast; ORM models in `data/storage/models.py`
NBA data	nba_api 1.11	Schedules, results, traditional and advanced box scores
Injury data	requests + pdfplumber 0.11	Downloading and parsing the official NBA injury-report PDFs
Market benchmark	The Odds API	Closing lines stored for comparison only (not inputs)
Charts (offline)	matplotlib, seaborn	Analysis figures
Website	Astro 5 (static), MDX, vanilla JS/CSS	660 pre-rendered pages, no client framework
Site tooling	Node 22, Playwright (screenshots/tests)	Build checks, UI tests, image rendering
Hosting	Cloudflare Pages + Wrangler 4	Static hosting at thehoopledger.com; one-command deploys
Legacy	Streamlit	Earlier dashboard (`dashboard/`), superseded by the website

Size: about **18,000 lines of Python** and **7,600 lines** of website code (Astro, JS, CSS, scripts) in 208 tracked files. Configuration comes from environment variables (`.env`, git-ignored): database host/port/name/user/password and the odds API key.

Why these choices

- **Gradient boosting** handles non-linear effects (a 10-point Elo edge matters more for a weak team than a strong one) and interactions without hand-built features, works well on a few thousand rows, and is exactly explainable with TreeSHAP.
- **PostgreSQL** gives one durable, queryable record of every forecast, which the grading and the website depend on.
- **A static site** is fast, cheap (free hosting), secure (no server, no database exposed) and reproducible: every page is a function of the exported data.

PART II · DATA

5 Data sources and the database

Sources

- **NBA Stats** (stats.nba.com, through the `nba_api` package): schedules, final scores, traditional and advanced box scores. Requests are spaced 1.65 s apart to respect rate limits. The site sometimes blocks cloud servers, which matters for automation (Section 23).
- **Official NBA injury report**: the league publishes a PDF every 15 minutes (hourly before 2025-26) at a predictable URL and keeps the archive online. The project downloads and parses each report it uses (pdfplumber), matches names to players, and stores every line. The same source feeds both live forecasts (latest report before now) and training (that day's 5:30 PM ET report), so the model trains on exactly what it will see live.
- **The Odds API**: closing spreads and totals, stored only to grade the model against the market.

Coverage

SEASON	FINAL GAMES	BOX SCORES	DAYS WITH AN INJURY REPORT
2021-22	1,317	1,317	210
2022-23	1,314	1,314	209
2023-24	1,312	1,312	204
2024-25	1,314	1,314	210
2025-26	1,314	1,314	207
2026-27 (in progress)	146	146	19

Main tables (row counts as of Nov 9, 2026)

TABLE	ROWS	HOLDS
teams	30	Team metadata
players	1,146	Player metadata
games	6,775	Schedule and results 2021-22 → 2026-27 (incl. upcoming)
team_box_scores	13,434	One row per team per game: traditional + advanced stats (ratings, pace, eFG%)
player_box_scores	167,495	One row per player per game
team_rolling_stats	53,932	5/7/10/20-game rolling team averages before each game
player_rolling_stats	155,084	5/10-game rolling player averages
injury_report_fetches	1,957	Each official injury-report PDF downloaded

TABLE	ROWS	HOLDS
injury_report_entries	97,297	Each player line in those reports (status, reason)
predictions	5,657	Every forecast saved, with inputs, timing and grade
game_odds	47	Stored market lines (benchmark only)

Box scores are never model inputs for the game they come from. They feed rolling averages for *later* games, the injury "rotation" definition, the explanations, and grading.

6 The 21 inputs (feature engineering)

Every input is known before tip-off. The same code builds them for training and for live forecasts, so the model never sees a definition at prediction time that it didn't learn from.

GROUP	INPUTS	DEFINITION
Team strength	`home_elo`, `away_elo`	Margin-of-victory Elo before the game (1500 = average)
	`elo_diff`	home Elo + 70 (home court) − away Elo
	`home/away_season_margin`	Season-to-date points margin per game, blended with last season for the first 20 games
	`home/away_games_played`	Games played this season (tells the model how far to trust the margin)
Recent form	`home/away_off_rating`	Points scored per 100 possessions, mean of last 10 games
	`home/away_def_rating`	Points allowed per 100 possessions, last 10 (lower is better)
	`combined_pace`	Average of the two teams' 10-game pace (possessions per 48 min)
Availability	`home/away_missing_pts`	Points per game of rotation players listed Out/Doubtful on the official report
Rest	`home/away_rest_days`	Days since previous game, capped at 5
	`home/away_back_to_back`	1 if second night of a back-to-back
Playoffs	`home_series_wins`, `is_elimination`, `series_pressure`	Series state (zero in the regular season)

Elo: one number per team

Every team starts at 1500. After each final game, the winner takes rating points from the loser. The size of the exchange depends on how surprising the result was and on the margin:

rating gap $d = \text{Elo}_{\text{home}} + 70 - \text{Elo}_{\text{away}}$ (70 Elo ≈ 2.5 points of home court)

expected home result $E = 1 / (1 + 10^{-d/400})$

margin multiplier $M = (|\text{margin}| + 3)^{0.8} / (7.5 + 0.006 \cdot d_{\text{winner}})$

update $\Delta = 20 \cdot M \cdot (\text{home won} - E)$ $\quad \text{Elo}_{\text{home}} \mathrel{+}= \Delta, \quad \text{Elo}_{\text{away}} \mathrel{-}= \Delta$

new season $\text{Elo} \leftarrow 1500 + 0.75 \cdot (\text{Elo} - 1500)$

The margin multiplier grows with the margin but with diminishing returns, and shrinks when a heavy favorite wins big, so strong teams can't farm rating points from blowouts. Empirically, about **28 Elo points ≈ 1 point of expected margin**. Ratings are replayed from the first game of 2021-22 (`ELO_START_DATE`); training and live use the identical `TeamStrength` class.

Season-margin blend

Elo is slow on purpose. For a faster read, each team's season-to-date average margin is added — but after 5 games that average is mostly noise, so it is blended with last season's margin: weight on this season = min(games, 20) / 20. After 20 games it is entirely this season.

Offensive and defensive rating, pace

Per-game ratings and pace come from the NBA's advanced box score (points per 100 possessions). The inputs are simple means over each team's previous 10 final games. A defensive rating is the opponents' offensive rating in those games.

Availability: missing rotation points

A team's **rotation** is every player who appeared in at least 2 of its last 10 games this season averaging 15+ minutes (and hasn't since played for another team). The input is the sum of those players' points per game for anyone listed **Out or Doubtful** on the official injury report. Questionable players are treated as available (they actually play about 49% of the time — see Section 13). For training, each historical game uses that day's 5:30 PM ET report; where no report is archived, box-score absences are used as a fallback and counted.

Rest and schedule

Days since each team's previous final game (home or away), capped at 5, plus a back-to-back flag. Schedule load (games in 4 or 6 nights), travel distance and time zones were tested and added nothing (Section 13).

7 The model

Two gradient-boosted regression models: one predicts the home team's points, the other the away team's points, from the same 21 inputs. Margin = home − away; total = home + away.

SETTING	VALUE	WHY
Algorithm	`GradientBoostingRegressor`	Captures non-linearities and interactions; exact TreeSHAP
Trees	150, learning rate 0.04	Many small steps generalise better than few large ones
Tree depth	3, min 15 samples per leaf	Shallow trees limit overfitting on ~6,700 games
Row / feature sampling	subsample 0.7, max_features 0.8	Stochastic boosting reduces variance
Recency weight	exp(−days ago / 550)	A game three seasons old counts ~13% as much as last night
Playoff weight	× 3	Playoffs are ~6% of games; weighting lifts them to ~16% of effective data
Training set	Every completed game since Oct 2021	More history matched or beat a 2023 start out of sample
Seed	random_state = 42	Reproducibility (see the caveat in Section 22)

Post-processing

The two predicted scores are converted to a total and a margin, clamped to sensible ranges (total 185–255, margin ±24), then rounded back to whole-number scores. If rounding produces a tie, the side favoured by a secondary "directional edge" signal gets the extra point. Stored forecasts are whole numbers.

What is deliberately *not* in the model

- **Betting lines.** Earlier versions pulled the total toward the Vegas total, floored the margin from the spread and broke ties with it. That made forecasts look better but made the model partly a copy of the market and impossible to grade against it. All of it sits behind `USE_MARKET_IN_PREDICTION = False`.
- **The player model.** A per-player points model exists and is still run for display, but its summed team totals came out 20–60 points low; blending it in cost correct winners in every test slice (65.5% vs 62.9% this season; 59.5% vs 54.8% in the playoffs). `TEAM_MODEL_ONLY = True`.
- **A direct margin model.** Trained and saved, but blending it 70/30 also cost winners, so it is off.

Confidence label

The pipeline also tags each forecast High / Medium / Low, mostly from the size of the projected margin. It is a rule of thumb, not a tested probability; the website shows win probabilities instead.

8 From margin to win probability

The win probability is a logistic curve on the projected home margin m (Platt scaling):

$$\text{P(home wins)} = 1 / (1 + e^{-(0.1197 \cdot m - 0.0247)})$$

The live curve's coefficients (0.1197, −0.0247) were fitted on an early, small set of graded forecasts; if no fitted curve is available the code falls back to $1 / (1 + e^{-m/9.5})$. In the walk-forward backtest each season uses a curve fitted only on earlier seasons' backtest forecasts. Refitting the live curve on the full record is on the roadmap.

PROJECTED HOME MARGIN	0	+2	+4	+6	+8	+10	+14
Home win probability	49%	55%	61%	67%	72%	76%	84%

So a 4-point favourite is about 61%, and a 10-point favourite about 77%. The probabilities are what get graded by the Brier score and the calibration chart (Section 11).

9 Forecast timing and the official record

Every forecast is saved with the time it was made and labelled. The rule (in `save_predictions`, Section 17.4):

- The **first** forecast saved for a game before tip-off becomes **official**.
- A rerun can replace it only within the **last hour before tip-off** (to pick up late injury news).
- Anything saved **after tip-off is never official**.
- Forecasts are never revised after the game.

The website shows one forecast per game: the issued forecast where the daily pipeline saved one (**65** games), otherwise the walk-forward backtest forecast (**5,325**). **168** forecasts from an older after-the-fact fill, which may have seen information from the games themselves, are excluded entirely. Ten source rows that list the same team as home and away are also excluded.

> **Be upfront about this in an interview**
>
> Most of the graded record is a **retrospective** backtest: forecasts produced after the games by models that had never seen them, not forecasts published beforehand. That is the standard way to evaluate on thousands of games, but it is different from a live track record. The live record grows every day (2026-27: issued forecasts plus backtest for games before the pipeline was running). Also, on Nov 8, 2026 several forecasts were marked official after tip-off at the author's request; the lock rule above is what automation will enforce.

10 How the model is tested

A few weeks of live forecasts can't tell you much: 30 games can show 70% or 55% by chance. To learn how good the model really is, it has to be tested on thousands of games — without letting it see anything it couldn't have known on the day.

Walk-forward backtest (`misc/walkforward_backfill.py`)

1. For each target season (2022-23 → 2026-27), train the score models **only on earlier seasons**, with the production settings. These models live in memory only; the production files are untouched.
2. Run every game of the target season through the **same function the daily pipeline uses** (`predict_single_game`), with that day's archived 5:30 PM ET injury report.
3. Fit the win-probability curve only on earlier seasons' backtest forecasts.
4. No betting lines. The player models (trained on all seasons) are switched off.
5. Save each row as `model_version='wf-v2'`, `prediction_type='walkforward'`, never official; grade it immediately.

Leakage guards

- Box-score stats from a game are never inputs to that game; rolling windows use only earlier games (`shift(1)` semantics).
- Availability uses the injury report published before the game, not who actually played.
- Elo and season margin are recorded *before* each game's result updates them.
- Before the backtest, three data gaps were closed: 209 days of 2022-23 injury reports, 260 missing box scores, and team form extended back to 2021.

Feature decisions: paired walk-forward cross-validation

Every candidate feature is tested by training with and without it on the same time-ordered folds, over three random seeds, and compared on margin error and winners picked (`model/experiment.py`). A feature is kept only if it helps consistently.

Metrics

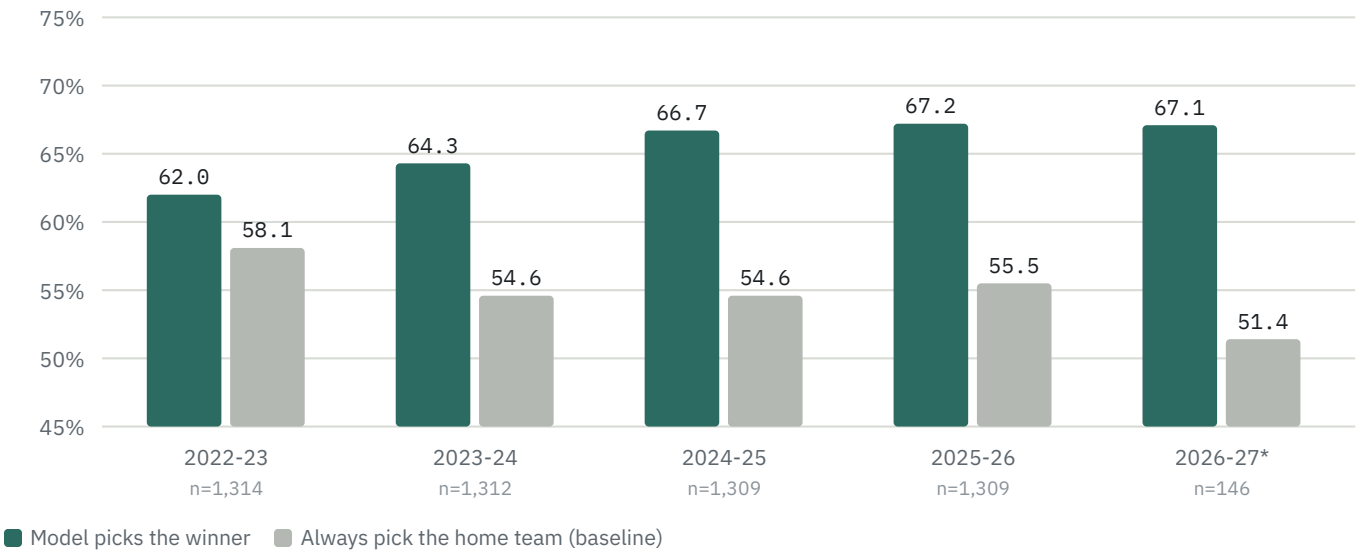
METRIC	DEFINITION	GOOD MEANS
Winners picked	Share of games where the team with the higher projected score won	Higher; baseline = home team 55.6%
Margin error (MAE)	Mean |forecast home margin − actual home margin|	Lower
Total error (MAE)	Mean |forecast total − actual total|	Lower
Brier score	Mean (home win probability − home won (1/0))²	Lower; always 50% scores 0.250
Calibration	Forecasts grouped by confidence: forecast vs share that won	Points on the diagonal

11 Results: all the numbers

5,390 graded games from 2022-10-18 to 2026-11-08 (65 issued + 5,325 walk-forward).

By season

SEASON	GAMES	WINNERS	HOME BASELINE	EDGE (PTS)	MARGIN MAE	TOTAL MAE	BRIER
2022-23	1,314	62.0%	58.1%	+3.9	10.34	14.92	0.231
2023-24	1,312	64.3%	54.6%	+9.7	11.25	15.08	0.223
2024-25	1,309	66.7%	54.6%	+12.1	10.98	14.88	0.211
2025-26	1,309	67.2%	55.5%	+11.7	11.44	15.21	0.206
2026-27 (in progress)	146	67.1%	51.4%	+15.7	11.73	15.38	0.208
All	**5,390**	**65.1%**	55.6%	+9.5	**11.02**	15.03	**0.217**



Winners picked by season vs always picking the home team. * season in progress.

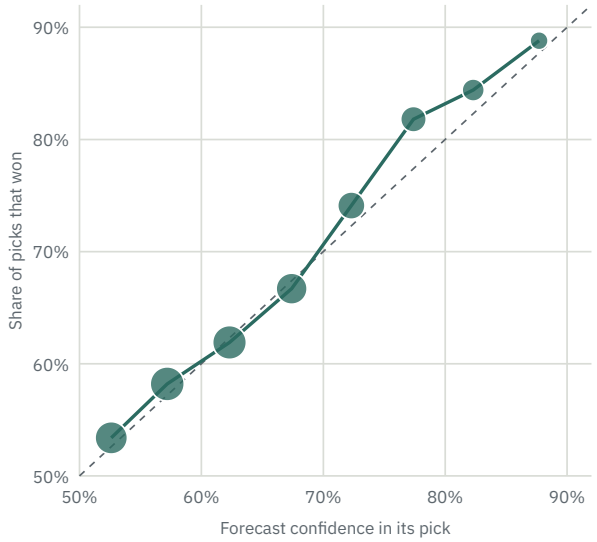
Accuracy rises from 2022-23 to 2025-26, partly because each later season has more history to train on. The average team-score miss is 9.4 points; half of team scores land within 8 points, 62% within 10.

Regular season vs playoffs

	GAMES	WINNERS	HOME BASELINE	MARGIN MAE	BRIER
Regular season	5,056	65.4%	55.4%	10.93	0.216
Playoffs	334	61.1%	57.8%	12.40	0.234

Playoffs are harder: evenly matched teams, adjustments within a series, and few training examples.

Calibration: are the probabilities honest?



CONFIDENCE	GAMES	FORECAST	WON	DIFF
50–55%	948	52.6%	53.4%	+0.8
55–60%	1,108	57.2%	58.2%	+1.0
60–65%	1,072	62.3%	61.9%	-0.4
65–70%	838	67.4%	66.7%	-0.7
70–75%	572	72.3%	74.1%	+1.8
75–80%	466	77.4%	81.8%	+4.4
80–85%	270	82.3%	84.4%	+2.1
85–100%	116	87.7%	88.8%	+1.1

When the model says 70%, the pick wins about 70% of the time. The curve sits on or slightly above the diagonal: at high confidence the favourite wins a little *more* often than forecast, i.e. the model is, if anything, mildly under-confident at the top.

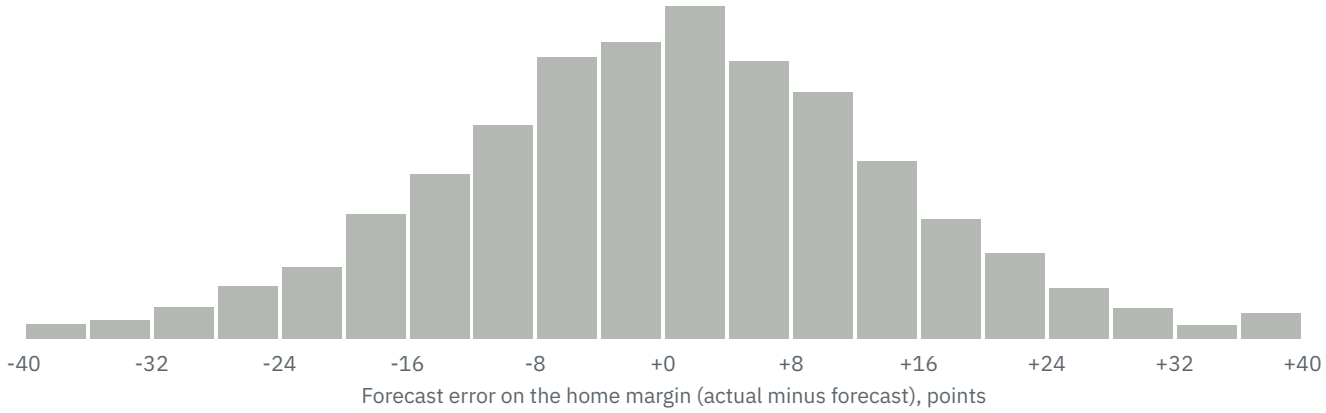
Accuracy by size of the projected margin

PROJECTED MARGIN	GAMES	WINNERS	MARGIN MAE	BRIER
0–3	1,954	54.9%	11.1	0.247
3–6	1,470	63.0%	11.2	0.234
6–10	1,167	72.2%	10.9	0.201
10+	799	83.7%	10.8	0.137

Is the margin itself unbiased?

FORECAST BUCKET	GAMES	AVG FORECAST MARGIN	AVG ACTUAL MARGIN
Road by 10+	205	-12.1	-12.0
Road by 6–10	512	-7.3	-7.1
Road by 3–6	636	-4.0	-3.8
Road by 0–3	651	-1.4	-0.6
Home by 0–3	1,076	+1.9	+1.8
Home by 3–6	891	+4.9	+4.1
Home by 6–10	766	+8.3	+8.0
Home by 10+	653	+13.6	+13.2

Within every bucket the average actual margin is close to the average forecast: the forecasts are unbiased on average, even though individual games scatter widely.



Distribution of margin errors. Standard deviation of actual margins 15.5; of forecast margins 6.8. Blowouts are rarely forecast as blowouts.

Against the betting market (benchmark only)

	MODEL	MARKET (CLOSING LINE)
Margin MAE	10.73	10.38
Total MAE	13.76	12.02
Same favourite	37 of 41	

On the 41 games with a stored line (2026-04-22 → 2026-11-08) the market is more accurate, as expected — it sees everything the model sees and more. The sample is far too small to conclude anything; the point is that the comparison is now fair.

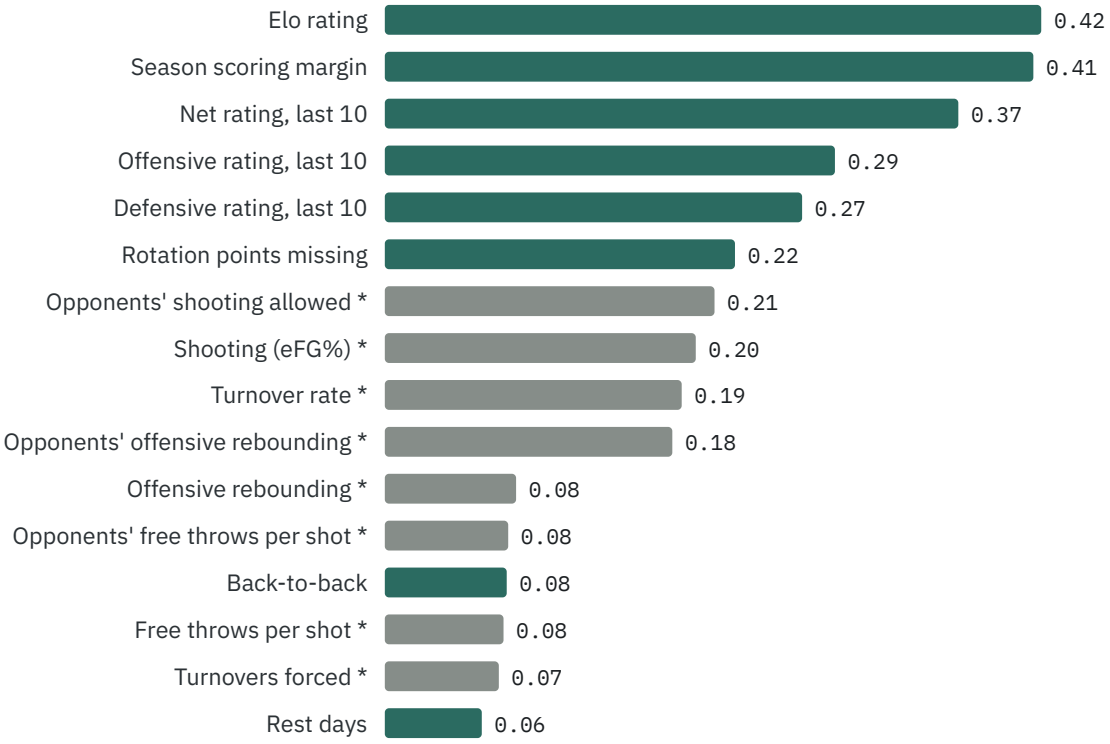
Most recent graded slate

Nov 8, 2026: **6–1** (average margin miss 9.1, Brier 0.125). The miss: the Lakers won at Detroit without Luka Dončić, whose absence was the largest single reason the model favoured Detroit.

12 What the numbers reveal

This is the answer to the motivating question. For each of 5,390 graded games, every pregame stat was turned into a home-minus-away gap and compared with the final margin.

Which pregame numbers line up with the result



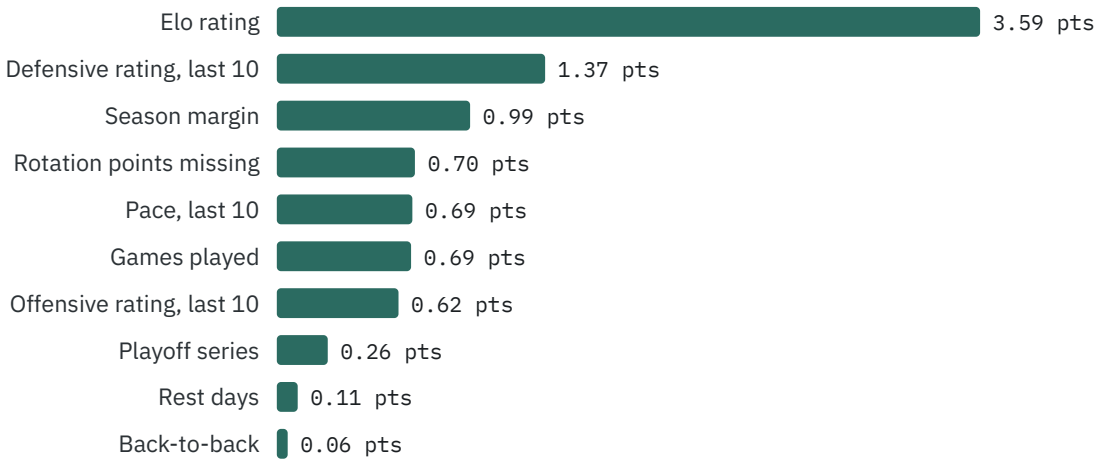
Correlation of each pregame gap with the final margin (signs set so “better” always points the same way). Teal: model inputs. Grey *: four-factor stats that are not inputs (10-game averages).

PREGAME GAP IN…	CORRELATION	BETTER SIDE WON
Elo rating	0.420	64.0%
Season scoring margin	0.415	63.4%
Net rating, last 10	0.367	62.9%
Offensive rating, last 10	0.288	60.0%
Defensive rating, last 10	0.267	59.2%
Rotation points missing	0.224	58.5%
Opponents' shooting allowed	0.211	57.0%
Shooting (eFG%)	0.199	57.7%
Turnover rate	0.190	56.3%
Opponents' offensive rebounding	0.184	56.3%

PREGAME GAP IN…	CORRELATION	BETTER SIDE WON
Offensive rebounding	0.084	50.5%
Opponents' free throws per shot	0.079	52.3%
Back-to-back	0.078	56.8%
Free throws per shot	0.076	52.4%
Turnovers forced	0.073	53.1%
Rest days	0.062	54.5%

- **Results-based measures win**: Elo (0.42), season margin and 10-game net rating beat any single box-score stat. They summarise everything at once.
- **Shooting matters most** among the four factors; offensive rebounding and free-throw rate barely separate winners from losers on their own.
- **Injuries matter**: the side missing fewer rotation points won 58.5%.
- **Back-to-backs are an open question**: the rested side won 56.8%, yet the model gives that input almost no weight; it is on the roadmap (Section 23). Some of that edge is home court, since road teams are on back-to-backs more often.
- Home teams won 55.6% regardless.

What the model actually leans on



Average absolute push on the projected margin per input group (exact TreeSHAP), over the 118 forecasts that can be explained. The model's priorities match the correlation ranking: team strength first.

What builds a rating: the four factors

A game's offensive rating can be split into Dean Oliver's four factors with one linear fit over 12,212 team-games ($R^2 = 0.947$, typical miss 2.7 points per 100 possessions):

$$\text{ORtg} \approx 31.7 + 151.2 \cdot \text{eFG\%} - 142.2 \cdot \text{TOV\%} + 44.8 \cdot \text{OREB\%} + 28.8 \cdot \text{FTr}$$

FACTOR	FORMULA	RATING PTS PER 1 PT OF FACTOR	TYPICAL SWING, OFFENSE	TYPICAL SWING, DEFENSE
Shooting	eFG% = (FG + 0.5·3P) / FGA	+1.51	±4.1	±4.2
Turnovers	TOV / (FGA + 0.44·FTA + TOV)	-1.42	±2.0	±2.1
Offensive rebounding	OREB / (OREB + opp DREB)	+0.45	±1.8	±1.7
Free throws	FTM / FGA	0.29 per 0.01	±0.9	±0.8

Shooting moves a team's rating about twice as much as anything else. These factors are *not* model inputs (a 42-feature version that included them did worse); they are used to explain where a team's rating comes from.

13 Experiments and decisions

Each idea is tested with the same walk-forward method before it reaches the live model. The results are written up in the website's Notebook and in comments next to the code they changed, so they aren't re-tested by accident.

EXPERIMENT	RESULT	DECISION
42 inputs → 21 lean inputs (Elo + season blend)	Walk-forward CV (3 seeds): margin MAE 12.05 → 11.65; winners 63.8% → 66.1%. Held-out 2026-27: MAE 12.63 → 11.64, winners 59.5% → 61.2%, Brier 0.244 → 0.223. Better on every seed.	Adopted
Blend the player model into game scores	Team model alone picked more winners in every slice: 65.5% vs 62.9% (2026-27), 75.2% vs 74.6% (Feb–Apr), 59.5% vs 54.8% (playoffs). Summed player totals ran 20–60 pts low.	Team model only
Direct margin model blended 70/30	Cost correct winners	Off
Betting lines inside the forecast	Made forecasts look better but circular; can't grade against the market	Removed; benchmark only
Train from 2021 instead of 2023	Held-out 2025-26: 65.3% → 67.0% winners; lower errors in 2 of 3 test seasons	Adopted (Nov 7)
Schedule load (games in 4/6 nights), travel miles, time zones, per-team home court	No improvement in margin error or winners	Rejected
Questionable players	Played in 49% of cases on the 5:30 PM report, yet counted as fully available	Open: treat as a probability
NBA Cup group games	180 games: scoring and late-game behaviour like regular games	No special handling
Does home court shrink vs better visitors?	Mostly no; road favourites finished near their forecast at every gap (2022-23 the exception)	No change

14 Explaining a forecast

Each game page answers “why does the model lean this way?” at three levels: the inputs it received, exactly how each input moved the forecast, and where each input came from in the box scores.

Exact contributions with TreeSHAP

For tree ensembles, SHAP's TreeExplainer gives **exact** additive contributions: base value + Σ contributions = model output. The published margin is home_model(x) − away_model(x), so the contribution of input *j* to the margin is home_shap[*j*] − away_shap[*j*], and the baseline is the difference of the two models' average outputs (≈ +2.1, the average home margin in training). Baseline plus all contributions equals the projected margin.

Only explain what can be verified

An explanation is only honest if it comes from the model that made the forecast. Stored scores are whole numbers, so the check is statistical: a model version is **verified** for a group of forecasts when ≥ 90% of them reproduce both stored scores to the point; a different model matches only ~12–19% by chance. Within a verified group, a forecast is explained if its margin matches within rounding. Currently **118** forecasts are explained (the current production model's 14, and 104 walk-forward forecasts from this season).

Built from the box scores

Every input is rebuilt from raw data and shown only if the rebuild reproduces the stored value exactly: Elo replayed game by game, season margin, the four-factor split of each 10-game rating (with an “everything else” remainder shown, not hidden), the injured players behind the missing points, and the previous game behind rest. Reproduction rates: Elo, season margin, missing points and rest 1,054/1,054; offensive rating 1,048; defensive 1,047; pace 513/527.

Plain-English reasons

A small rule-based writer (`web/analytics/narrative.py`) turns the verified numbers into sentences such as “Their defense holds opponents to 49% eFG against a 53% league average, which alone adds about 6 points per 100 possessions.” It is deterministic and unit-tested (41 tests), not a language model, so it can never state a number that isn't in the data.

> **Caveat worth stating**
>
> SHAP explains how the model used its inputs, not what caused the result. A large Elo contribution means the model leaned on Elo, not that Elo decided the game.

PART IV · THE WEBSITE

15 The website

A static site built with Astro from JSON files exported out of the database. There is no server and no database connection at runtime; every page is pre-rendered.

Pages

PAGE	WHAT IT SHOWS
Home	Headline record (LED scoreboard), last night's results and tonight's games as scoreboard cards (records, hot/cold streaks, countdowns, upset tags), biggest surprise of the week, data status line
Forecasts	Calendar of every game by day/week, filters by team and status, daily pick strip, coin-flip and 'Why' reasons
Game pages (~570)	Forecast vs result, why the model leans one way, inputs and SHAP waterfall, box-score breakdown, injury report, recent form, head-to-head
Teams	Elo trend, upcoming and recent games, predictability (how hard the team is to forecast), surprise games
Surprises	Largest upsets and largest margin misses
What wins	Which pregame stats line up with winning, what the model leans on, four-factor swings, home court
Performance	Full evaluation: by season, calibration, winner calls, last 30 picks, errors, market benchmark
Notebook / Methodology / About	Research write-ups (MDX), full method, project background

Build pipeline

Refreshing the site

```
python web/scripts/export_data.py         # DB -> web/src/data/*.json, web/public/data/archive-*.json
python web/scripts/explain_forecasts.py   # TreeSHAP per forecast -> explanations.json
python web/scripts/breakdown_inputs.py    # raw box-score rebuilds + narratives -> breakdowns.json
python web/scripts/what_wins.py           # correlation study -> what_wins.json
cd web && npm run build                   # Astro -> web/dist (660 pages)
node scripts/check_build.mjs              # recomputes metrics from the archive, checks explanations
npx wrangler pages deploy dist --project-name thehoopledger --branch main
```

Design and front-end

- Light editorial pages with dark “instrument” panels (scoreboards, charts). Fonts: Source Serif 4, IBM Plex Sans and Mono (self-hosted). One orange accent; teal marks “the model”; team colours from a lookup.
- All charts are hand-built SVG in Astro components (no chart library). Motion is progressive enhancement: an IntersectionObserver adds `.in` to reveal charts, count up numbers, roll final scores like an odometer and stamp ✓/✗; nothing is hidden without JavaScript, and `prefers-reduced-motion` disables it all.

- Client JavaScript is minimal: the Forecasts calendar, the homepage slate choice (by US Eastern date), tip-off countdowns.
- `check_build.mjs` independently recomputes every headline number from the per-game archive and re-adds every explanation, failing the build on any mismatch.

Hosting

Cloudflare Pages project `thehoopledger`, deployed by direct upload with Wrangler (the code repository stays private). Every deploy is kept and can be rolled back in one click. Custom domains thehoopledger.com and www.

PART V · THE CODE

16 Repository layout

Top-level structure

```
NBA-Predictor/
├── config/settings.py          Season, DB connection (from .env), API settings, rate limit
├── data/
│   ├── ingestion/              Everything that gets data in and builds features
│   │   ├── fetch_games.py          schedule + results (marks final only when W/L is set)
│   │   ├── fetch_box_scores.py     traditional + advanced box scores
│   │   ├── fetch_official_injuries.py  official injury PDFs: download, parse, store
│   │   ├── fetch_injuries.py       ESPN injury feed (live fallback only)
│   │   ├── fetch_odds.py           market lines (benchmark only)
│   │   ├── fetch_players.py / fetch_teams.py   metadata
│   │   ├── compute_rolling_stats.py        5/7/10/20-game team averages
│   │   ├── compute_player_rolling_stats.py 5/10-game player averages
│   │   ├── team_strength.py        Elo + season-margin blend (shared by training and live)
│   │   ├── build_features.py       assembles the training table (availability, rest, ...)
│   │   └── update_series.py etc.   playoff series state
│   └── storage/
│       ├── db.py                   SQLAlchemy engine
│       └── models.py               ORM schema (14 tables)
├── model/
│   ├── train.py                    FEATURE_COLS, training, evaluation, save .pkl
│   ├── predict.py                  daily prediction path (features -> scores -> save/lock)
│   ├── calibrate.py                win-probability curve (Platt)
│   ├── evaluate_predictions.py     grade forecasts against finals
│   ├── experiment.py               paired walk-forward CV for feature decisions
│   ├── train_players.py / predict_players.py   player model (display only)
│   └── *.pkl                       saved models (home, away, margin, calibration)
├── pipeline/run_daily.py       One command for the daily cycle (modes below)
├── misc/                       One-off and maintenance scripts
│   ├── walkforward_backfill.py     the backtest
│   ├── make_official.py            mark latest forecasts official
│   └── backfill_rebounds.py        fill missing OREB/DREB
├── web/                        The website
│   ├── src/pages/              routes: index, forecasts, games/[id], teams, ...
│   ├── src/components/         Astro components (cards, charts, breakdowns)
│   ├── src/lib/                data access, streaks, reasons, motion, colours
│   ├── src/data/               exported JSON (generated)
│   ├── src/content/notebook/   research write-ups (MDX)
│   ├── analytics/              Python: SHAP, four factors, narratives, metrics (+ tests)
│   ├── scripts/                export, explain, breakdowns, what-wins, build checks
│   └── public/                 static files, per-season archives, og.png, favicon
├── analysis/, visuals/         earlier feature-importance studies and charts
├── dashboard/, site/           legacy Streamlit dashboard and old static page
└── requirements.txt            Python dependencies (see Section 22)
```

17 Programming deep dives

The excerpts below are copied verbatim from the repository at the time of writing.

17.1 Elo update — `data/ingestion/team_strength.py`

One class serves training and live forecasting. Training walks every final game in date order, recording `features()` before calling `update()` with the result, so each game's inputs only reflect earlier games. Live forecasting replays all finals before the date and then reads the state.

TeamStrength.update

```
def update(self, home_id, away_id, season, home_score, away_score):
        self._roll_season(home_id, season)
        self._roll_season(away_id, season)
        margin = float(home_score) - float(away_score)
        dr = self.elo[home_id] + ELO_HOME - self.elo[away_id]
        exp = 1 / (1 + 10 ** (-dr / 400))
        dr_winner = dr if margin > 0 else -dr
        mov_mult = ((abs(margin) + 3) ** 0.8) / (7.5 + 0.006 * dr_winner)
        delta = ELO_K * mov_mult * ((margin > 0) - exp)
        self.elo[home_id] += delta
        self.elo[away_id] -= delta
        for tid, mg in ((home_id, margin), (away_id, -margin)):
            st = self.season_stats.setdefault((tid, season), [0, 0.0])
            st[0] += 1
            st[1] += mg
```

TeamStrength.features — season-margin blend and Elo gap

```
def features(self, home_id, away_id, season):
        rec = {}
        for side, tid in (('home', home_id), ('away', away_id)):
            self._roll_season(tid, season)
            rec[f'{side}_elo'] = self.elo[tid]
            n, s = self.season_stats.get((tid, season), (0, 0.0))
            w = min(n, BLEND_GAMES) / BLEND_GAMES
            cur = s / n if n else 0.0
            rec[f'{side}_season_margin'] = w * cur + (1 - w) * self.last_season_margin.get(tid, 0.0)
            rec[f'{side}_games_played'] = n
        rec['elo_diff'] = rec['home_elo'] + ELO_HOME - rec['away_elo']
        return rec
```

17.2 Training — `model/train.py`

train_model: recency and playoff weighting, model settings

```
def train_model(train, target_col):
    """Train a GradientBoosting model for one score target."""
    X_train = train[FEATURE_COLS]
    y_train = train[target_col]

    # Recency weighting — slower decay over 2 years
    max_date = pd.to_datetime(train['game_date']).max()
    days_ago = (max_date - pd.to_datetime(train['game_date'])).dt.days
    sample_weights = np.exp(-days_ago / 550)

    # Playoff games are only ~6% of training data but are what we predict in the postseason.
    # 3x multiplier brings their effective share from ~6% to ~16%.
    is_playoff = (train['season_type'] == 'Playoffs').astype(float)
    sample_weights = sample_weights * (1.0 + 2.0 * is_playoff)

    model = GradientBoostingRegressor(
        n_estimators=150,
        learning_rate=0.04,
        max_depth=3,
        min_samples_leaf=15,
        subsample=0.7,
        max_features=0.8,
        random_state=42
    )

    model.fit(X_train, y_train, sample_weight=sample_weights)
    return model
```

Running `python model/train.py` holds out the most recent 15% of games chronologically, prints MAE / winner accuracy / train-vs-test overfitting checks and plots, then **refits on every game** and saves the production models.

17.3 One forecast — `model/predict.py`

`predict_single_game` builds the features (`build_game_features` + `add_live_availability`), gets the two scores, runs the (now inactive) ensemble and the post-processing, computes confidence and key factors, and stores the exact input vector with the forecast. A guard refuses to predict if any input is missing rather than silently using 0:

get_team_predictions

```
def get_team_predictions(model_home, model_away, row_dict, model_margin=None):
    """Get predictions from team models (and optionally the direct margin model)."""
    missing = [f for f in FEATURE_COLS
               if row_dict.get(f) is None or pd.isna(row_dict.get(f))]
    if missing:
        # Used to become 0 silently, which the model never saw in training
        raise ValueError(
            f"{row_dict.get('away_team')} @ {row_dict.get('home_team')}: "
            f"missing features {missing}")
    features = np.array([float(row_dict[f]) for f in FEATURE_COLS]).reshape(1, -1)

    home_pred = float(model_home.predict(features)[0])
    away_pred = float(model_away.predict(features)[0])

    direct_margin = None
    if model_margin is not None:
        direct_margin = float(model_margin.predict(features)[0])

    return home_pred, away_pred, direct_margin
```

17.4 The official-forecast lock — `save_predictions`

Who becomes official

```
is_before_tipoff = True
is_within_lock_window = False

if tipoff:
    is_before_tipoff = now_utc < tipoff
    is_within_lock_window = (
        tipoff - timedelta(hours=1)
        <= now_utc
        < tipoff
    )

is_official = False
lock_reason = "snapshot_only"
prediction_type = "debug"

if is_backfill:
    # Backfill: mark as official pre-game simulation unless one already exists
    if existing_official is None:
        is_official = True
        lock_reason = "backfill"
        prediction_type = "backfill"
    else:
        lock_reason = "backfill_duplicate"
        prediction_type = "backfill"
elif is_before_tipoff:
    if existing_official is None:
        is_official = True
        lock_reason = "first_game_day_prediction"
        prediction_type = "daily"
    elif is_within_lock_window:
        existing_official.is_official = False
        is_official = True
        lock_reason = "pre_tipoff_lock_window_override"
        prediction_type = "pre_tip"
    else:
        lock_reason = "non_official_rerun"
        prediction_type = "rerun"
else:
    lock_reason = "after_tipoff_not_official"
    prediction_type = "post_tip"
```

17.5 Rotation and missing points — `build_features.py`

compute_live_availability

```
def compute_live_availability(team_id, game_date, out_names):
    """
    Availability for an unplayed game: rotation players (same definition as training)
    listed Out/Doubtful on the injury report.
    Returns (missing_min, missing_pts, missing_player_names).
    """
    if not out_names:
        return 0.0, 0.0, []
    from data.ingestion.fetch_official_injuries import season_for_date

    box = _load_played_box_scores(before_date=game_date)
    tb = box[box.team_id == team_id]
    if tb.empty:
        return 0.0, 0.0, []

    last_team_by_date = box.sort_values('game_date')[['player_id', 'game_date', 'team_id']]
    gdate = pd.Timestamp(game_date)
    rot = _season_rotation(tb, season_for_date(gdate.date()), gdate, team_id,
                           last_team_by_date)

    out = {normalize_player_name(n) for n in out_names}
    absent = rot[rot['full_name'].map(normalize_player_name).isin(out)]
    return (float(absent['mins'].astype(float).sum()), float(absent['pts'].sum()),
            absent['full_name'].tolist())
```

17.6 Margin attribution — `web/analytics/explain.py`

Exact TreeSHAP contributions to the margin

```
def margin_attribution(model_home, model_away, X: np.ndarray):
    """Exact TreeSHAP for both score models on complete rows of X.

    Returns dict with home/away/margin contributions (n x k), baselines, and the
    raw model outputs; raises if additivity fails."""
    import shap
    eh, ea = shap.TreeExplainer(model_home), shap.TreeExplainer(model_away)
    sh, sa = eh.shap_values(X), ea.shap_values(X)
    bh, ba = float(np.ravel(eh.expected_value)[0]), float(np.ravel(ea.expected_value)[0])
    yh, ya = model_home.predict(X), model_away.predict(X)
    err = max(np.abs(bh + sh.sum(1) - yh).max(initial=0), np.abs(ba + sa.sum(1) - ya).max(initial=0))
    if err > 1e-4:
        raise AssertionError(f"SHAP additivity failed: max error {err:.2e}")
    return dict(home=sh, away=sa, margin=sh - sa, base_home=bh, base_away=ba,
                base_margin=bh - ba, pred_home=yh, pred_away=ya, pred_margin=yh - ya)
```

Verifying that a model version made a group of forecasts

```
def verify_group(pred_home, pred_away, stored_ph, stored_pa) -> tuple[bool, float]:
    """(verified, share of forecasts reproducing both scores)."""
    m = score_match(pred_home, pred_away, stored_ph, stored_pa)
    share = float(m.mean()) if len(m) else 0.0
    return share >= VERIFY_SHARE, share
```

17.7 Four factors — `web/analytics/breakdown.py`

Dean Oliver's four factors

```
def four_factors(fgm, fga, fg3m, fta, ftm, tov, oreb, opp_dreb):
    """Four factors for one team-game (or arrays of them). Fractions."""
    fgm, fga, fg3m, fta, ftm, tov, oreb, opp_dreb = (
        np.asarray(v, dtype=float) for v in (fgm, fga, fg3m, fta, ftm, tov, oreb, opp_dreb))
    with np.errstate(divide="ignore", invalid="ignore"):
        return {
            "efg": (fgm + 0.5 * fg3m) / fga,
            "tov": tov / (fga + 0.44 * fta + tov),
            "orb": oreb / (oreb + opp_dreb),
            "ftr": ftm / fga,
        }
```

Splitting a rating gap into factor parts + remainder

```
def split_rating(rating: float, factors: dict, league_rating: float,
                 league_factors: dict, coef: dict) -> dict:
    """Split (rating - league_rating) into one part per factor plus 'other'.
    Parts are in rating points (per 100 possessions) and sum exactly to the gap."""
    parts = {k: coef[k] * (factors[k] - league_factors[k]) for k in FACTORS}
    gap = rating - league_rating
    parts["other"] = gap - sum(parts.values())
    return parts
```

17.8 Stale-data guard — `pipeline/run_daily.py`

Refuse to forecast from stale data

```
def step_check_previous_games_final(retries=2):
    """Before predicting: every team playing today must have its previous game
    final. If not (e.g. the results fetch failed quietly), re-fetch results, box
    scores and today's rolling stats, then check again; fail loudly if still stale."""
    import time
    stale = _stale_teams()
    for attempt in range(retries):
        if not stale:
            return
        log.info(f"  Previous game not final for {', '.join(f'{t} ({d})' for t, d in stale)}"
                 f" — re-fetching (attempt {attempt + 1}/{retries})")
        if attempt:
            time.sleep(30)
        step_fetch_games_today()
        step_fetch_box_scores()
        step_rolling_stats_today()
        step_player_rolling_stats_today()
        stale = _stale_teams()
    if stale:
        raise RuntimeError(
            "stale inputs: previous game not final for "
            + ", ".join(f"{t} ({d})" for t, d in stale)
            + ". Their forecasts would miss that game (Elo, rest, rolling stats).")
```

17.9 Live games are not final — `data/ingestion/fetch_games.py`

The NBA endpoint returns live, partial scores for games in progress. An early version stored those as finals. The fix requires the W/L field, which is only set when a game is over:

_rows_are_final

```
def _rows_are_final(game_rows):
    """LeagueGameFinder returns live games with partial PTS; WL is only set
    once the game is over, so require WL and PTS on both team rows."""
    if len(game_rows) < 2 or 'WL' not in game_rows.columns:
        return False
    return bool(game_rows['WL'].notna().all()
                and game_rows['PTS'].notna().all())
```

18 Setting it up from scratch

1. **Prerequisites**: Python 3.12+ (developed on 3.14), PostgreSQL 15+ (developed on 18), Node 20+ (developed on 22), Git.
2. **Clone** the repository and create a virtual environment:

```
git clone https://github.com/MounirArayssi/NBA-Predictor.git
cd NBA-Predictor
python -m venv .venv
.venv\Scripts\activate           # Windows  (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
pip install scikit-learn shap matplotlib seaborn   # not all are listed yet; see Section 22
```

3. **Configure** a `.env` file in the project root (never commit it):

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=nba_predictor
DB_USER=postgres
DB_PASSWORD=...
ODDS_API_KEY=...        # optional: only for the market benchmark
```

4. **Database**: create the database, then either restore a backup (`psql -d nba_predictor -f backup.sql`) or create tables from the ORM and backfill with the fetch scripts (games, box scores, injury reports for each season). A full backfill is slow because of API rate limits.
5. **Train**: `python model/train.py` (writes `model/*.pkl`).
6. **Website**: `cd web && npm install`, then the refresh sequence in Section 15. Preview with `npm run dev` (http://localhost:4321) or `npm run preview` after a build.
7. **Deploy access**: `npx wrangler login` once (opens a browser to authorise the Cloudflare account).

19 Daily operations runbook

Until automation is built, a day looks like this (US Eastern times). Run from the project root with the virtual environment active. On Windows terminals, set `PYTHONIOENCODING=utf-8` if output is piped.

Morning: grade last night, refresh the site

Morning

```
# 1. results + box scores (no forecasting, no retrain)
python -c "from pipeline.run_daily import run_step, step_fetch_games_today, step_fetch_box_scores; \
run_step('games', step_fetch_games_today); run_step('box', step_fetch_box_scores)"

# 2. grade the slate
PYTHONPATH=. python -c "from datetime import date; from model.evaluate_predictions import
evaluate_predictions; \
evaluate_predictions(start_date=date(2026,11,8), end_date=date(2026,11,8))"

# 3. refresh and publish (Section 21)
```

Pre-game: forecast tonight

Evening

```
# after the 5:30 PM ET injury report, before the first tip-off
python pipeline/run_daily.py --today-only --skip-twitter
PYTHONPATH=. python misc/make_official.py          # only if the run didn't lock them itself
# then refresh and publish the site
```

`run_daily.py` modes

COMMAND	WHAT IT DOES
`run_daily.py`	Full cycle: fetch all games, box scores, odds, full rolling stats, player rolling stats, series → retrain → stale check → predict → tweet (dry run)
`--today-only`	Only what today needs: recent results + today's schedule, today's rolling rows; skips retraining
`--skip-retrain`	Full data refresh without retraining
`--predict-only`	Stale check + predict, no fetching
`--skip-twitter` / `--post-twitter`	Skip the tweet step / actually post (default is a dry run)

20 Retraining and changing the model

Routine retrain

The full daily run retrains on all history each morning so the model includes last night's games. To retrain by hand: `python model/train.py`. It evaluates on a chronological 15% holdout first (read the printed MAE, winner accuracy and the train-vs-test lines to spot overfitting), then refits on everything and overwrites `model/model_home.pkl`, `model_away.pkl`, `model_margin.pkl` and `feature_cols.pkl`. Back up the old .pkl files first if you might want to roll back.

Changing a feature or a setting: the protocol

1. Write down the hypothesis (what new information the input adds that Elo/form/availability don't).
2. Build it in `build_features.py` **and** in the live path (`predict.py` / `team_strength.py`) with the same definition. Training/live mismatch is the most common silent bug.
3. Test with paired walk-forward CV (`model/experiment.py`), 3 seeds, on margin MAE, winners and Brier. Keep it only if it helps consistently.
4. Add it to `FEATURE_COLS`, retrain, re-run the walk-forward backtest (`misc/walkforward_backfill.py`; delete the old `wf-v2` rows or use a new model_version), re-export the site.
5. Record the result next to `FEATURE_COLS` and in a Notebook entry, including failures.

Other model maintenance

- **Win-probability curve**: `python model/calibrate.py` refits the Platt curve on all graded official forecasts (needs ≥ 20). Planned: refit on the full record.
- **Elo start date** (`ELO_START_DATE`) and constants must only change together with a retrain; training and live must replay the same history.
- **New season**: update `CURRENT_SEASON` in `config/settings.py`; Elo carries 75% over automatically; the season-margin blend handles the small sample.

21 Updating and deploying the website

Data refresh (after new results or forecasts)

```
python web/scripts/export_data.py
python web/scripts/explain_forecasts.py      # see the caveat below
python web/scripts/breakdown_inputs.py
python web/scripts/what_wins.py
cd web
npm run build
node scripts/check_build.mjs                 # must print "All build checks passed"
npx wrangler pages deploy dist --project-name thehoopledger --branch main
```

Code or design changes

1. Edit in `web/src/`; preview live with `npm run dev`.
2. Python analytics changes: run the unit tests: `PYTHONPATH=web python -m unittest discover -s web/analytics/tests -t web`.
3. Build, run `check_build.mjs`, check pages at desktop and phone width, then deploy with the command above.
4. Commit and push: `git add -A && git commit -m "..." && git push` (work happens on `development`; merge to `main` with a pull request).

Rollback

Cloudflare dashboard → Workers & Pages → thehoopledger → Deployments → pick a previous deployment → **Rollback**. Instant, no rebuild.

Changing the brand or domain

Site name: `TITLE` in `web/src/lib/data.js`. Domain: `site` in `web/astro.config.mjs` (or the `SITE_URL` environment variable). Share image: re-render `web/scripts/og-image.html` to `web/public/og.png` at 1200×630.

22 Maintainability: known issues and gotchas

ISSUE	SYMPTOM	FIX / STATUS
Training is not fully deterministic	Retrains on unchanged data can differ; walk-forward explanations stop reproducing (this season's verified share dropped from 94% to ~20% after a DB change)	Training sorts only by `game_date`; with `subsample=0.7` the row order of same-day games changes the model. Fix: sort by (game_date, game_id), then re-run the backtest. Until then, previously verified explanations are merged back after `explain_forecasts.py`.
Stale forecasts	A forecast made before a team's previous game was recorded (wrong rest, stale Elo)	Fixed: `step_check_previous_games_final` re-fetches and stops the run
Live scores stored as final	In-progress games marked final with partial scores	Fixed: require W/L on both team rows
Encoding crash	UnicodeEncodeError on emoji when output is piped (Windows)	Fixed in run_daily; set `PYTHONIOENCODING=utf-8` elsewhere
Scripts moved to `misc/`	ModuleNotFoundError	Run with `PYTHONPATH=.` from the project root
Missing offensive rebounds	2024-25 and 2025-26 box scores lack OREB/DREB for many games	`misc/backfill_rebounds.py` (resumable); ~611 games remain. Breakdowns fold rebounding into 'everything else' until filled
Same team home and away	10 source rows can't be graded	Excluded everywhere, with a warning in the exporter
`requirements.txt`	UTF-16 encoded and missing scikit-learn, shap, matplotlib, seaborn	Regenerate as UTF-8 with `pip freeze`
README	Describes the older ensemble (player blend, Vegas anchoring, 50+ features)	Update to the current design (this guide)
NBA Stats blocking	Timeouts from cloud servers	Run the pipeline from a home machine

Health checks to run regularly

- `check_build.mjs` on every build (recomputes all published metrics from raw rows).
- Unit tests in `web/analytics/tests`.
- The homepage status line: results date, latest forecast, injury report used, model trained, last update. Anything older than a day is a red flag.
- Database backups (`pg_dump`) before migrations or bulk backfills.

23 Limitations and roadmap

Limitations

- Most of the graded record is a retrospective backtest, not forecasts published in advance.
- Forecast margins are much narrower than real ones; blowouts are rarely predicted as blowouts.
- Questionable players count as available; late scratches after the forecast are missed.
- A missing player is measured by points per game, not his full impact (defence, on/off).
- The market remains more accurate on the small sample with lines.
- The site is rebuilt by hand today.

Roadmap

AREA	NEXT STEPS
Automation	Morning: results → grade → retrain → export → build → check → deploy. ~1 h before each tip: injury report → forecast → lock → publish. Runs on a home machine (Task Scheduler); never publishes partial data; alerts on failure.
Model prerequisites	Deterministic training sort; strict pre-tip lock with no manual post-tip officials.
Model ideas	Questionable as a probability; on/off player impact instead of points; margin + total models instead of two score models; refit the probability curve on the full record; revisit back-to-backs (57% for the rested side vs ~0 weight).
Data	Finish the rebound backfill; historical odds to study whether the market over-reacts to schedule spots.
Website	Player stats (teased as coming soon), team-page form strips, 404 and robots.txt, accessibility and speed passes.

APPENDICES

A Explaining it in an interview

A two-minute walkthrough

1. **Problem**: forecast every NBA game before tip-off and be graded publicly.
2. **Data**: NBA Stats box scores and the official injury-report PDFs into PostgreSQL.
3. **Features**: 21 pregame inputs — Elo with margin of victory, season margin, 10-game ratings, injured rotation points, rest, playoff state.
4. **Model**: two gradient-boosted regressors for home and away points; logistic curve for win probability.
5. **Evaluation**: walk-forward backtest across 5,390 games: 65.1% winners vs 55.6% baseline, Brier 0.217, well calibrated.
6. **Product**: a static Astro site with exact SHAP explanations and box-score breakdowns, on Cloudflare.
7. **Lessons**: fewer features, no market leakage, honest grading.

Likely questions

Why gradient boosting instead of a neural network or linear regression?

About 6,700 games and 21 tabular inputs: boosted trees are the strong default for that. They capture non-linear effects and interactions without hand-made features, train in seconds, and are exactly explainable with TreeSHAP. A linear model is a useful baseline but misses interactions; a neural net would overfit this little data.

Why predict two scores instead of the winner directly?

Scores carry more information than a win/loss label (a 20-point win and a 1-point win are different), give the margin and total for free, and the margin maps to a calibrated probability. A direct margin+total model is on the roadmap to test.

How do you prevent data leakage?

Every input is computed from games strictly before the forecast; Elo and season margin are recorded before each game updates them; availability uses the injury report published before the game; the backtest trains only on earlier seasons and replays each day's report through the live code path.

How do you know the probabilities are any good?

Brier score 0.217 vs 0.250 for a coin flip, and the calibration table: in every confidence band the share of picks that won is within a few points of the forecast.

What surprised you?

How little the extra statistics added. Cutting from 42 to 21 inputs improved every metric. And how noisy single games are: actual margins vary more than twice as much as forecasts do.

Why not use betting lines? They'd make it more accurate.

They would, but the model would become a copy of the market and could no longer be measured against it. Lines are stored only as a benchmark.

What would you do with more time?

Automate the daily cycle with safety checks, make training deterministic, model Questionable players as probabilities, and measure injuries by on/off impact.

What was the hardest engineering problem?

Making training and live forecasting compute identical inputs: the same Elo replay, the same rotation definition, the same injury source. Plus honest grading — the lock rule, excluding after-the-fact forecasts, and only explaining forecasts whose model can be verified.

B Glossary

TERM	MEANING
Elo	A rating system where teams exchange points after each game according to how surprising the result was.
Margin / total	Home score − away score / home + away score.
Offensive / defensive rating	Points scored / allowed per 100 possessions.
Pace	Possessions per 48 minutes.
eFG%	Effective field-goal %: (FG + 0.5·3P) / FGA; credits threes for their extra point.
Four factors	Shooting, turnovers, offensive rebounding, free throws: Dean Oliver's decomposition of efficiency.
Gradient boosting	An ensemble that adds many small decision trees, each correcting the previous ones' errors.
MAE	Mean absolute error: the average size of a miss.
Brier score	Mean squared error of a probability against the 0/1 outcome; 0.25 = always 50%.
Calibration	Whether events forecast at X% happen about X% of the time.
Platt scaling	A logistic regression that turns a score (here the margin) into a probability.
Walk-forward backtest	Testing each period with a model trained only on data before it.
Leakage	Accidentally letting a model use information it wouldn't have had at prediction time.
SHAP / TreeSHAP	Additive per-feature contributions to one prediction; exact and fast for tree models.
Rotation player	Here: 2+ of the team's last 10 games this season, 15+ minutes per game.
Static site	Pre-rendered HTML files served as-is, with no application server.

C Numbers cheat sheet

NUMBER	VALUE
Games graded	5,390 (2022-10-18 → 2026-11-08)
Winners picked	65.1%
Home-team baseline	55.6%
Margin MAE / total MAE / team-score MAE	11.02 / 15.03 / 9.42 pts
Brier score	0.217 (coin flip 0.250)
2026-27 record	98–48 (67.1%)
Best season	2025-26: 67.2% winners, Brier 0.206
Playoffs	61.1% winners over 334 games
Projected margin 10+	83.7% correct
Projected margin < 3	54.9% correct
SD of actual vs forecast margins	15.5 vs 6.8
Inputs	21 (from 42)
Trees per model	150, depth 3, lr 0.04
Elo constants	K 20, home +70, carry 0.75, ~28 Elo per point
Recency time constant	550 days; playoffs ×3
Win-probability curve	P = 1/(1+e^−(0.1197m − 0.0247))
Best single predictor	Elo gap, r = 0.42
Four-factor fit	R^2 0.947 over 12,212 team-games
Market benchmark (41 games)	margin MAE 10.73 model vs 10.38 market
Database	6,775 games · 167,495 player box scores · 97,297 injury-report lines · 5,657 predictions
Website	660 pages · ~72 MB · Astro 5 · Cloudflare Pages